

Bisher:

iterative Algorithmen

- mehrfaches Durchlaufen von Programmschritten mit Hilfe von Schleifen
- verwenden oft Akkumulator-Variablen zur schrittweisen Berechnung des Ergebnisses (z.B. res)

Jetzt:

rekursive Algorithmen

Idee: Führe das Problem für x zurück auf das Problem für kleineren Wert als x .

Bsp: mathematische Definition der Fakultät:

$$fak(x) = \begin{cases} x \cdot fak(x-1) & , \text{ falls } x > 1 \\ 1 & , \text{ sonst} \end{cases}$$

Rekursive Definition, denn "fak" tritt in ihrer eigenen Definition auf.

$$\begin{aligned} fak(3) &= 3 \cdot fak(2) \\ &= 3 \cdot 2 \cdot fak(1) \\ &= 3 \cdot 2 \cdot 1 \\ &= 6 \end{aligned}$$

- Lösung eines Problems durch einen rekursiven Algorithmus ist oft völlig analog zur Problemstellung
($\Rightarrow$ deklarative Programmierung
 $\Rightarrow$ funktionale Programmierung)
- Rekursive Methoden: rekursiver Aufruf mit Argumenten, die "kleiner" als die ursprüngliche Eingabe sind.

Für Terminierung: Man muss irgendwann nicht-rekursiven Fall erreichen.

Klassifikation verschiedener Arten von Rekursion

- linearer / nicht-lineare Rekursion
 - linear: in jeder Ausführung des Methodenaufrufs höchstens ein rekursiver Aufruf (Bsp: fak)
 - nicht-linear: Bsp fib

Bsp: Fibonacci-Zahlen

Ziel: Vorhersage von Kaninchen-Populationen

Vereinfachende Annahmen:

- Kaninchen werden nach 1 Monat geschlechtsreif.
- Tragezeit: 1 Monat
- Bei jeder Geburt wird ein Kaninchenpaar geboren.
- Kaninchen sind unsterblich.

- Kaninchen sind unsterblich.

$f(x)$ = Anzahl der Kaninchenpaare im Monat x .

Alg. hat nicht-lineare Rekursion, ist aber sehr ineffizient:

$$\begin{aligned}
 f(20) &= f(19) + f(18) \\
 &= f(18) + f(17) + f(18) \\
 &= f(17) + f(16) + f(17) + f(17) + f(16) \\
 &= \dots
 \end{aligned}$$

$f(19)$ wird 1-mal aufgerufen
 $f(18)$ wird 2-mal aufgerufen
 $f(17)$ wird 3-mal — — —
 $f(16)$ wird 5-mal — — —
 $f(15)$ wird 8-mal — — —

Zur Berechnung von $f(n)$ müssen "in etwa" 2^n Berechnungsschritte ausgeführt werden.

(Exponentieller Aufwand).

Es ginge auch mit linearem Aufwand.

- direkte/verschränkte Rekursion

direkte Rekursion: Algorithmus ruft sich selbst wieder rekursiv auf (z.B. fak, f)

verschränkte Rekursion: z.B. f ruft g auf,
(mutual recursion) g ruft f auf

• Endrekursion (tail recursion)

Spezialfall der direkten Rekursion, bei dem die rekursiven Aufrufe nur am Ende des Alg. auftreten.

D.h.: weder in Teilausdrücken noch vor weiteren Anweisungen des Alg.

Bsp: fak ist nicht endrekursiv, denn nach Beendigung des rek. Aufrufs muss man den ursprünglichen Wert von x noch kennen und damit multiplizieren.